The dataset provided below contain the 18 days workout sessions of a person which contains, duration of workout, date, and the calories burnt**.**

data.csv

Duration	Date	Calories
60	2020/12/01	420
45	2020/12/02	380
NaN	12/03/2020	390
450	2020/12/04	450
30	2020/14/05	NaN
60	2020/12/06	300
45	not_a_date	250
60	2020/12/06	300
60	2020/12/08	NaN
30	2020/12/09	410

Answer the programming questions below based on the dataset provided

1.

A)How do you remove all rows that contain any empty cells in a DataFrame?

Answer:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.dropna(inplace=True)
print(df.to_string())
```

B) Create a line plot showing how the number of calories burned changes over dates. Make sure to handle missing values and fix the date format.

Answer:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('data.csv')
# Clean the data
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
df.dropna(subset=['Date', 'Calories'], inplace=True)
# Line plot
plt.figure(figsize=(8, 4))
plt.plot(df['Date'], df['Calories'], marker='o', color='blue')
plt.title('Calories Burned Over Time')
plt.xlabel('Date')
plt.ylabel('Calories')
plt.grid(True)
plt.tight_layout()
plt.show()
```

2.

A) How do you detect rows where the value in the Calories column is missing?

Answer:

```
import pandas as pd
df = pd.read_csv('data.csv')
missing = df[df["Calories"].isnull()]
print(missing)
```

B)Classify durations into categories: Short (≤45), Medium (=60), and Long (>60). Plot their proportions using a pie chart.

Answer:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('data.csv')
def classify(duration):
    if duration <= 45:
        return 'Short'
    elif duration == 60:
        return 'Medium'
    else:
        return 'Long'
df.dropna(subset=['Duration'], inplace=True)
df['Category'] = df['Duration'].apply(classify)
# Pie chart
category_counts = df['Category'].value_counts()
category_counts.plot(kind='pie', autopct='%1.1f%%', startangle=90, colors=['skyblue',
'orange', 'pink'])
plt.title('Proportion of Duration Categories')
plt.ylabel('')
plt.show()
```

3.

A)If a Duration value is clearly wrong (for example, 450 minutes among mostly 30–60), how can you cap values above 120 to 120?

Answer:

```
import pandas as pd
df = pd.read_csv('data.csv')
for x in df.index:
    if df.loc[x, "Duration"] > 120:
        df.loc[x, "Duration"] = 120
print(df.to_string())
```

B)Draw a bar chart showing the average calories burned for each unique Duration value.

Answer:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('data.csv')
df.dropna(subset=['Duration', 'Calories'], inplace=True)
grouped = df.groupby('Duration')['Calories'].mean()
# Bar chart
grouped.plot(kind='bar', color='green')
plt.title('Average Calories Burned vs Duration')
plt.xlabel('Duration (minutes)')
plt.ylabel('Average Calories')
plt.tight_layout()
plt.show()
```

4.

A)How can you identify and then remove duplicate rows in the DataFrame?

Answer:

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.duplicated())
df.drop_duplicates(inplace=True)
print(df.to_string())
```

B)Create a scatter plot to show the relationship between Duration and Calories.

Answer:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv('data.csv')
df.dropna(subset=['Duration', 'Calories'], inplace=True)

sns.scatterplot(x='Duration', y='Calories', data=df)
plt.title('Duration vs Calories')
plt.show()
```

5.

A) How can you fill empty cells in a specific column, say Calories, with its mean value?

Answer:

```
import pandas as pd
df = pd.read_csv('data.csv')
cal_mean = df["Calories"].mean()
df.fillna({"Calories": cal_mean}, inplace=True)
print(df.to_string())
```

B)Plot a histogram to show how often each Duration appears.

Answer:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv('data.csv')
df.dropna(subset=['Duration'], inplace=True)
sns.histplot(df['Duration'], bins=5)
plt.title('Duration Frequency')
plt.show()
```

6.

A) How do you convert a column with mixed-format dates into proper datetime objects and then remove the rows where the date is invalid?

Answer:

```
import pandas as pd
df = pd.read_csv('data.csv')
# Convert 'Date' to datetime format (invalid formats become NaT)
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
# errors='coerce' converts wrong formats to NaT.
# Remove rows where 'Date' is NaT (invalid)
df.dropna(subset=['Date'], inplace=True)
print(df.to_string())
```

B)How would you remove all rows where Duration **exceeds 120 minutes?**

Answer:

```
import pandas as pd

df = pd.read_csv('data.csv')
for x in df.index:
  if df.loc[x, "Duration"] > 120:
    df.drop(x, inplace=True)

print(df.to_string())
```

UNIT-3

1. The $2-\sqrt{2}$ can be computed as the root of the function $f(x)=x2-2f(x)=x2-2$. Starting at $a=0a=0$ and $b=2b=2$, use *my_bisection* to approximate the $2-\sqrt{2}$ to a tolerance of $|f(x)|<0.1|f(x)|<0.1$ and $|f(x)|<0.01|f(x)|<0.01$. Verify that the results are close to a root by plugging the root back into the function**.(BISECTION)**

```
import numpy as np

def my_bisection(f, a, b, tol):
    if np.sign(f(a)) == np.sign(f(b)):
        raise Exception(
            "The scalars a and b do not bound a root")
    m = (a + b) / 2
    if np.abs(f(m)) < tol:
        return m
    elif np.sign(f(a)) == np.sign(f(m)):
                return my_bisection(f, m, b, tol)
    elif np.sign(f(b)) == np.sign(f(m)):
                return my_bisection(f, a, m, tol)

    f = lambda x: x ** 2 - 2
    r1 = my_bisection(f, 0, 2, 0.1)
    print("r1 =", r1)
    r01 = my_bisection(f, 0, 2, 0.01)
    print("r01 =", r01)

    print("f(r1) =", f(r1))
    print("f(r01) =", f(r01))
```

2. Use *my_newton*= to compute 2–√2 to within tolerance of 1e-6 starting at *x0* = *1.5*.**(NETWON-RAPHSON)**

```
import numpy as np

f = lambda x: x**2 - 2
f_prime = lambda x: 2*x
newton_raphson = 1.4 - (f(1.4))/(f_prime(1.4))

print("newton_raphson =", newton_raphson)
print("sqrt(2) =", np.sqrt(2))
def my_newton(f, df, x0, tol):
    # output is an estimation of the root of f
    # using the Newton Raphson method
    # recursive implementation
    if abs(f(x0)) < tol:
        return x0
    else:
        return my_newton(f, df, x0 - f(x0)/df(x0), tol)

estimate = my_newton(f, f_prime, 1.5, 1e-6)
print("estimate =", estimate)
print("sqrt(2) =", np.sqrt(2))
```

3. Use the Trapezoid Rule to approximate ∫π0sin(x)dx∫0πsin⁡(x)dx with 11 evenly spaced grid points over the whole interval. Compare this value to the exact value of 2.**(Trapezoid Rule)**

```
import numpy as np

a = 0
b = np.pi
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)

I_trap = (h/2)*(f[0] + \
         2 * sum(f[1:n-1]) + f[n-1])
err_trap = 2 - I_trap

print(I_trap)
print(err_trap)
```

UNIT-2

STACK

```
# Initialize stack
stack = []

# Stack operations
def push():
    item = input("Enter item to push: ")
    stack.append(item)
    print(f"Pushed: {item}")

def pop():
    if is_empty():
        print("Stack Underflow! Cannot pop.")
    else:
        print(f"Popped: {stack.pop()}")

def peek():
    if is_empty():
        print("Stack is empty.")
    else:
        print(f"Top of stack: {stack[-1]}")

def display():
    if is_empty():
        print("Stack is empty.")
    else:
        print("Stack (Top to Bottom):", list(reversed(stack)))
def is_empty():
    return len(stack) == 0
def size():
    print(f"Stack size: {len(stack)}")
```

```
# Menu loop
while True:
    print("\n--- Stack Operations Menu ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display Stack")
    print("5. Stack Size")
    print("6. Exit")

    choice = input("Enter your choice (1-6): ")

    if choice == '1':
        push()
    elif choice == '2':
        pop()
    elif choice == '3':
        peek()
    elif choice == '4':
        display()
    elif choice == '5':
        size()
    elif choice == '6':
        print("Exiting program.")
        break
    else:
        print("Invalid choice. Please enter a number from 1 to 6.")
```

QUEUE

```
# Initialize an empty queue
queue = []

def is_empty():
    return len(queue) == 0

def enqueue(item):
    queue.append(item)  # Add item at the end

def dequeue():
    if is_empty():
        return "Queue is empty"
    return queue.pop(0)  # Remove item from the front

def peek():
    if is_empty():
        return "Queue is empty"
    return queue[0]  # Front item

def size():
    return len(queue)

def display():
    print("Queue:", queue)

# Example usage
enqueue(10)
enqueue(20)
enqueue(30)
display()
```

```
print("Dequeued:", dequeue())
display()

print("Front element:", peek())
print("Queue size:", size())
```

LINEAR SEARCH

```
#write a program to implement linear search using recursion
def linerec(arr, key, index=0):
    if index >= len(arr):
        return -1
    if arr[index] == key:
        return index
    return linerec(arr, key, index + 1)# recurssion

arr = [5, 8, 2, 9, 1]
key = 9
result = linerec(arr, key) #function call
if result != -1:
    print(f"Element found at index {result}")
else:
     print("Element not found")
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Element found at index 3
```

BINARY SEARCH

```
#Program to implement Binary Search using recursion.
def bs(arr, key, low, high):
    if low > high:
        return -1  # Base case: target not found

    mid = (low + high) // 2

    if arr[mid] == key:
        return mid  # Target found
    elif arr[mid] < key:
        return bs(arr, key, mid + 1, high)  # Search right half
    else:
        return bs(arr, key, low, mid - 1)  # Search left half

arr = [1,3,5,7,9,11,13]
key = 2
result = bs(arr, key,0,len(arr)-1)
if result!= -1:
    print(f"the element {key} found at index {result}")
else:
     print(f" the element {key} not found")
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
the element 13 found at index 6
```

BUBBLE SORT

```
#write Program / function to implement Bubble Sort
def bubble(arr):
    n = len(arr)
    for i in range(n):
        swapped = False  # Flag to detect any swap
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap if elements are in the wrong order
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        # If no two elements were swapped by inner loop, then break
        if not swapped:
            break

arr = [64, 34, 25, 12, 22, 11, 90]
print("Original Array:", arr)

bubble(arr)
print("Sorted Array:", arr)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Original Array: [64, 34, 25, 12, 22, 11, 90]
Sorted Array: [11, 12, 22, 25, 34, 64, 90]
```

How do you create a list in Python and perform the following operations?

Create a list

Add elements to the list

Remove elements from the list

Update elements in the list

```
my_list = [10, 20, 30]
print("Initial List:", my_list)
# Adding elements
my_list.append(40)
my_list.extend([50, 60])
my_list.insert(2, 25)
print("After Additions:", my_list)
# Removing elements
my_list.remove(30)
popped = my_list.pop()
print("Popped Element:", popped)
my_list.pop(1)
del my_list[0]
print("After Removals:", my_list)
# Updating elements
my_list[1] = 99
print("After Update:", my_list)
# Clearing all elements
my_list.clear()
print("After Clear:", my_list)
```

Write a Python program to create a tuple of five numbers. Then:

1. **Print the first and last elements.**
2. **Find and print the length of the tuple.**
3. **Try changing the second element and observe what happens.**

```
# Create a tuple of numbers
numbers = (10, 20, 30, 40, 50)

# Print first and last elements
print("First element:", numbers[0])
print("Last element:", numbers[-1])

# Print length of the tuple
print("Length of tuple:", len(numbers))

# Try changing the second element (this will raise an error)
numbers[1] = 25  #  Tuples are immutable, so this will cause a TypeError
```

Write a Python program to create a set of fruits. Add a new fruit to the set, remove one fruit, and then check if a specific fruit (e.g., "Apple") is present in the set.

```
# Create a set of fruits
fruits = {"Apple", "Banana", "Mango"}

# Add a new fruit
fruits.add("Orange")

# Remove one fruit
fruits.remove("Banana")  # Make sure "Banana" exists

# Check for membership
if "Apple" in fruits:
    print("Apple is in the set.")
else:
    print("Apple is not in the set.")

# Print final set
print("Current set of fruits:", fruits)
```

Write a Python program to create a dictionary storing names of students as keys and their marks as values. Add a new student, update an existing student's marks, and print all student names with their marks.

```
# Create dictionary of students and their marks
student_marks = {
    "A": 85,
    "B": 90,
    "C": 78
}

# Add a new student
student_marks["D"] = 92

# Update marks for an existing student
student_marks["B"] = 95

# Print all students and their marks
print("Student Marks:")
for name, marks in student_marks.items():
    print(f"{name}: {marks}")
```

UNIT-1

Write a Python program to find the factorial of a given number.

```
# Input from user
num = int(input("Enter a number: "))

# Initialize factorial result
factorial = 1

# Check if the number is negative
if num < 0:
    print("Factorial is not defined for negative numbers.")
elif num == 0 or num == 1:
    print("Factorial of", num, "is 1")
else:
    for i in range(2, num + 1):
        factorial *= i
    print("Factorial of", num, "is", factorial)
```

Write a Python program to print the Fibonacci sequence up to n terms.

```
# Input: number of terms
n = int(input("Enter the number of terms: "))
# First two Fibonacci numbers
a, b = 0, 1
# Print Fibonacci sequence
print("Fibonacci sequence:")
for i in range(n):
    print(a, end=' ')
    a, b = b, a + b
```

Write a Python program to input three numbers and print them in ascending order without using the .sort() method.

```
# Input 3 numbers from the user
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

# Compare and reorder using conditional logic
if a > b:
    a, b = b, a  # swap a and b
if a > c:
    a, c = c, a  # swap a and c
if b > c:
    b, c = c, b  # swap b and c

# Print in ascending order
print("Numbers in ascending order:", a, b, c)
```